Module: concept_library/

Concept & Curriculum System

Position in Architecture

```
concept_library  ◄──── this module
    │
    ├──► llm_orchestrator   (reads ConceptPage, NamedEquation to build prompts)
    ├──► explanation_bus    (reads CurriculumPath for navigation state)
    └──► server/session     (reads concept_ref from IR to route scenes)
```

This module has no dependencies on any other new module. It is a pure data store — it defines structures and provides read-only query methods. Nothing in this module performs IO, calls the LLM, or produces DSL. It is the authoritative source of mathematical knowledge in the system.

Directory Layout

```
concept_library/
├── mod.rs            # Public API, LibraryStore, re-exports
├── concept.rs        # Concept, TermAnnotation, ConceptCategory
├── equation.rs       # NamedEquation, DerivationStep, EquationRole
├── page.rs           # ConceptPage, ConceptSection, SectionKind
├── curriculum.rs     # CurriculumPath, UserProgress
├── prerequisite.rs   # PrerequisiteGraph — DAG of concept dependencies
└── query.rs          # LibraryQuery — search and navigation helpers
```

File: concept_library/concept.rs

Purpose

Defines Concept — the atomic unit of mathematical knowledge — and the annotation types that describe what each term in an equation means.

What it contains

`Concept` struct:

- `id: String` — unique identifier. Format: snake_case topic path, e.g. `"calculus.derivatives.chain_rule"`, `"trig.unit_circle"`, `"linalg.eigenvalues"`.
- `title: String` — human-readable title: "The Chain Rule", "Unit Circle", "Eigenvalues and Eigenvectors".
- `category: ConceptCategory` — top-level classification.
- `prerequisite_ids: Vec<String>` — ordered list of concept IDs that must be understood before this one. These form the prerequisite DAG edges. Order matters: they are listed from most foundational to most immediate.
- `intuition: String` — a 2–4 sentence plain-English paragraph explaining what the concept is and why it matters, written without symbols. This is the first text shown to the user.
- `formal_definition_latex: String` — the LaTeX string for the formal mathematical definition. Rendered in the Narrative Panel header.
- `equation_ids: Vec<String>` — ordered list of `NamedEquation` IDs associated with this concept. Order is pedagogical: from the definition through corollaries.
- `scene_ids: Vec<String>` — DSL scene file identifiers that visualize this concept. These are the pre-authored fallback scenes; LLM-generated scenes augment but do not replace them.
- `misconceptions: Vec<String>` — common mistakes or confusions. Shown in a collapsible "Watch out for" section.
- `related_concept_ids: Vec<String>` — cross-references to related concepts for non-linear exploration.
- `difficulty: DifficultyLevel` — Beginner, Intermediate, Advanced.
- `estimated_minutes: u32` — rough estimate for the frontend to show "~12 min" on the curriculum navigator.

`ConceptCategory` enum:

- `Arithmetic`, `Algebra`, `Trigonometry`, `Calculus`, `LinearAlgebra`, `DifferentialEquations`, `Statistics`, `NumberTheory`, `AbstractAlgebra`, `Topology`, `DiscreteMathmatics`

`DifficultyLevel` enum:

- `Beginner`, `Intermediate`, `Advanced`

`TermAnnotation` struct — describes what one sub-expression in a `NamedEquation` represents:

- `node_id: String` — the `AnnotatedExpr.node_id` from `dsl/ast.rs` this annotation applies to. This is the link between the DSL expression tree and the human-readable explanation.

- `symbol_latex: String` — the LaTeX for this term as it appears in the equation, e.g. `"f'(g(x))"`.
- `name: String` — short plain-English name: "outer derivative", "inner function", "exponent".
- `tooltip: String` — 1–2 sentence explanation shown on hover: "This is the derivative of the outer function f, evaluated at g(x), not at x directly."
- `highlight_token: Option<String>` — if this term has been assigned a highlight token by the LLMOrchestrator, it is stored here so the concept library can provide it to the prompt builder.

File: `concept_library/equation.rs`

Purpose

Defines `NamedEquation` — a named, fully annotated mathematical statement with a derivation chain.

What it contains

`NamedEquation` struct:

- `id: String` — unique identifier. Format: `"concept_id.equation_name"`, e.g. `"calculus.derivatives.chain_rule.statement"`, `"calculus.derivatives.chain_rule.proof_step_1"`.
- `concept_id: String` — the parent concept this equation belongs to.
- `display_name: String` — shown in the Equation Panel header: "Chain Rule Statement", "Chain Rule — Step 1: Identify Outer and Inner Functions".
- `latex: String` — the full LaTeX string for the equation as it appears in the display. This is the "whole equation" rendered at the top of Panel B.
- `role: EquationRole` — classifies the equation's purpose.
- `term_annotations: Vec<TermAnnotation>` — one entry per significant sub-expression. The LLMOrchestrator reads these to build prompts. The frontend reads them to render tooltips.
- `derivation_chain: Vec<String>` — ordered list of `NamedEquation` IDs leading to this equation. Empty for axioms and definitions; non-empty for theorems. The UI "Derivation" section shows these as previous steps.
- `numeric_example: Option<NumericExample>` — concrete numbers substituted into the equation to demonstrate it, pre-authored for fallback display.
- `dsl_expression_id: Option<String>` — if this equation has a corresponding `MathExpression` node in a DSL file, this is its `node_id`. Links the catalog entry to the live runtime evaluation.

`EquationRole` enum:

- `Definition` — introduces a new concept
- `Theorem` — a proven result
- `Formula` — a computational recipe
- `Identity` — always true (e.g., $\sin^2+\cos^2=1$)
- `Approximation` — an approximation with known error bounds
- `Lemma` — an intermediate result used to prove something else
- `DerivationStep` — one step in a multi-step derivation (not standalone)

`NumericExample` struct:

- `variable_values: Vec<(String, f64)>` — e.g., `[("x", 2.0), ("g", "sin")]` (for function substitutions, the value is the function's output at the given input)
- `result_value: f64` — the final numerical answer
- `step_results: Vec<f64>` — intermediate numerical results at each derivation step

`DerivationStep` struct (used within `NamedEquation.derivation_chain` resolution):

- `equation_id: String` — the `NamedEquation` ID for this step
- `transition_description: String` — one sentence describing how we got from the previous step to this one: "Apply the chain rule: differentiate the outer function $\sin^2$ and keep the inner function x^2 unchanged."

File: `concept_library/page.rs`

Purpose

Defines `ConceptPage` — a complete lesson unit — and `ConceptSection` — one section within a page. These are the structures the LLMOrchestrator reads to build its prompts.

What it contains

`ConceptSection` struct:

- `id: String` — unique within the page, e.g. `"intuition"`, `"definition"`, `"chain_rule_derivation"`, `"worked_example_1"`, `"practice_1"`.
- `kind: SectionKind` — type classification.
- `title: String` — displayed as a subheading in Panel A.
- `body_text: String` — the prose for this section. May contain equation references using the template syntax `{{eq:equation_id}}` which the Narrative Panel resolves to inline KaTeX. May contain term reference

syntax `{{term:node_id}}` which the panel resolves to a highlighted span.

- `equation_ids: Vec<String>` — `NamedEquation` IDs to display as block equations within this section, in order.
- `scene_id: Option<String>` — the DSL scene identifier that Panel C should load when this section is active. `None` means use the current scene unchanged.
- `step_range: Option<(usize, usize)>` — if this section corresponds to a range of steps in the equation derivation (e.g., section "Apply Chain Rule" covers steps 3–7), specifies the start and end step indices. Used by the `AnimationSyncController` to seek the timeline.
- `practice_problem: Option<PracticeProblem>` — if `kind == Practice`, contains the problem statement and hidden solution.

`SectionKind` enum:

- `Intuition` — informal introduction, no symbols
- `FormalDefinition` — LaTeX definition with term annotations
- `Derivation` — step-by-step proof or derivation
- `WorkedExample` — a specific example worked through numerically
- `Visualization` — focuses on what the scene is showing
- `Practice` — problem for the user to attempt
- `Summary` — key takeaways

`PracticeProblem` struct:

- `statement: String` — the problem text with LaTeX
- `hint: String` — shown on user request
- `solution_steps: Vec<String>` — hidden solution steps shown on reveal
- `numeric_answer: Option<f64>` — if the answer is a number, for automatic checking

`ConceptPage` struct:

- `concept_id: String` — the concept this page teaches
- `sections: Vec<ConceptSection>` — in the order they should be presented
- `current_section_index: usize` — runtime navigation state (not persisted in the catalog; managed by the session)
- `estimated_minutes: u32` — carried from `Concept.estimated_minutes`
- `version: u32` — content version, incremented when sections are updated

---

File: `concept_library/curriculum.rs`

Purpose

Defines `CurriculumPath` — an ordered sequence of `ConceptPage` references forming a course — and `UserProgress` — the per-user completion state.

What it contains

`CurriculumPath` struct:

- `id: String` — e.g. `"intro_calculus"`, `"linear_algebra_ml"`.
- `title: String` — "Introduction to Differential Calculus".
- `description: String` — 2–3 sentences shown on the course selection screen.
- `concept_ids: Vec<String>` — the ordered list of concept IDs in this path. Order determines the curriculum sequence.
- `prerequisite_path_ids: Vec<String>` — other curriculum paths that should be completed first.
- `level: DifficultyLevel` — overall difficulty.
- `estimated_hours: f32` — total time estimate.
- `domain: String` — "Mathematics", "Physics", "Machine Learning", etc.

`UserProgress` struct:

- `path_id: String`
- `completed_concept_ids: HashSet<String>` — concepts fully completed
- `current_concept_id: String` — where the user currently is
- `current_section_index: usize` — which section within the current concept
- `current_step_index: usize` — which derivation step within the current section
- `section_visit_log: Vec<(String, String, u64)>` — `(concept_id, section_id, unix_timestamp)` for analytics
- `practice_scores: HashMap<String, bool>` — `(problem_id, correct)` for practice problems

Methods on `UserProgress`:

- `advance_section(&mut self, page: &ConceptPage)` — moves to the next section, marks current as visited.
- `advance_concept(&mut self, path: &CurriculumPath)` — marks current concept complete, moves to next.
- `can_access_concept(&self, concept_id: &str, graph: &PrerequisiteGraph) -> bool` — checks whether all prerequisites are met.

- `completion_percentage(&self, path: &CurriculumPath) -> f32` — for the progress bar.

File: `concept_library/prerequisite.rs`

Purpose

Defines `PrerequisiteGraph` — the directed acyclic graph of concept dependencies. Enables the UI to lock inaccessible concepts and guide users toward prerequisites they are missing.

What it contains

`PrerequisiteGraph` struct:

- `edges: HashMap<String, Vec<String>>` — maps `concept_id` to its prerequisite concept IDs. This is the adjacency list of the DAG.
- `reverse_edges: HashMap<String, Vec<String>>` — maps a concept to all concepts that depend on it. Used to show "this unlocks..." information.

Methods:

- `build(concepts: &[Concept]) -> Self` — constructs both edge maps from the concepts' `prerequisite_ids` fields.
- `prerequisites_of(&self, concept_id: &str) -> &[String]` — direct prerequisites.
- `all_prerequisites_of(&self, concept_id: &str) -> Vec<String>` — transitive closure (DFS), returns all ancestors.
- `unlocks(&self, concept_id: &str) -> &[String]` — concepts that become accessible after completing this one.
- `validate(&self) -> Result<(), Vec<String>>` — cycle detection using DFS. Returns a list of cycle descriptions if any are found. Called at library initialization.
- `topological_sort(&self) -> Vec<String>` — returns all concepts in a valid learning order. Used to validate that `CurriculumPath.concept_ids` is topologically valid.

File: `concept_library/query.rs`

Purpose

Provides `LibraryQuery` — a collection of search and navigation helper methods over the library data. Keeps query logic out of the data structures themselves.

What it contains

`LibraryQuery` struct:

- `store: Arc<LibraryStore>` — shared reference to the library data.

Methods:

- `concept_by_id(&self, id: &str) -> Option<&Concept>`
- `equation_by_id(&self, id: &str) -> Option<&NamedEquation>`
- `page_for_concept(&self, concept_id: &str) -> Option<&ConceptPage>`
- `section_by_id(&self, concept_id: &str, section_id: &str) -> Option<&ConceptSection>`
- `equations_for_concept(&self, concept_id: &str) -> Vec<&NamedEquation>` — in the order defined by `Concept.equation_ids`.
- `derivation_chain_for(&self, equation_id: &str) -> Vec<&NamedEquation>` — resolves the full chain from axiom to this equation.
- `term_annotation_for_node(&self, concept_id: &str, node_id: &str) -> Option<&TermAnnotation>` — finds the annotation for a specific AST node ID within a concept's equations.
- `search_concepts(&self, query: &str) -> Vec<&Concept>` — simple substring search on title and intuition text. Not semantic search — for that, the LLM is used directly.
- `next_concept_in_path(&self, path_id: &str, current_concept_id: &str) -> Option<&Concept>`
- `prev_concept_in_path(&self, path_id: &str, current_concept_id: &str) -> Option<&Concept>`

File: `concept_library/mod.rs`

Purpose

`LibraryStore` — the central store for all library data — and public API.

What it contains

`LibraryStore` struct:

- `concepts: HashMap<String, Concept>`
- `equations: HashMap<String, NamedEquation>`
- `pages: HashMap<String, ConceptPage>`
- `paths: HashMap<String, CurriculumPath>`
- `prerequisite_graph: PrerequisiteGraph`

`LibraryStore::load_from_directory(path: &Path) -> Result<Self, LibraryError>` — loads all JSON/TOML content files from a directory. Each concept is one file. This is the primary initialization path for the server.

`LibraryStore::load_builtin() -> Self` — loads a minimal built-in library compiled into the binary (for development/demo). Includes at minimum: Unit Circle, Chain Rule, Integration by Parts, Matrix Multiplication, Eigenvectors.

`LibraryStore::query(&self) -> LibraryQuery` — returns a query helper bound to this store.

`LibraryError` enum:

- `FileNotFound(PathBuf)`
- `ParseError { file: PathBuf, reason: String }`
- `CycleDetected(Vec<String>)`
- `MissingEquation { concept_id: String, equation_id: String }`
- `InvalidNodeReference { equation_id: String, node_id: String }`

Content File Format

Each concept is stored as a TOML file in a `concepts/` directory. TOML is preferred over JSON for human-authored content because it supports multi-line strings cleanly (for `intuition`, `body_text`, `formal_definition_latex`). `NamedEquation` objects are stored in a `equations/` subdirectory.

Dependencies Summary

Depends on	Why
Standard library	Storage, collections
`serde` + `toml`	Content file deserialization
`dsl/ast.rs`	`node_id` type reference (conceptual, not import)

Consumed by

Consumer	What it reads
`llm_orchestrator/`	`Concept`, `NamedEquation`, `ConceptPage`, `ConceptSection`, `TermAnnotation` — all used to build LLM prompts

Consumer	What it reads
`explanation_bus/`	`CurriculumPath`, `UserProgress` — for curriculum navigation state
`server/session.rs`	`LibraryStore.query()` — to route a loaded scene's `concept_ref` to the correct `ConceptPage`
Frontend curriculum navigator	`CurriculumPath`, `UserProgress.completion_percentage()`